

# 大语言模型的指纹

杨志广, 上海大学, 上海 200444, 中国; 邮箱: yangzg@shu.edu.cn

吴瀚舟, 上海大学, 上海 200444, 中国; 邮箱: h.wu.phd@ieee.org (通讯作者)

## Abstract

近期进展证实, 大语言模型 (LLMs) 能够在各种任务上达到最先进的性能。然而, 由于从头训练 LLMs 需要大量资源, 保护 LLMs 的知识产权免受侵权变得至关重要且紧迫。这促使本文作者提出一种新颖的 LLMs 黑盒指纹技术。我们首先证明, LLMs 的输出跨越了一个与每个模型相关联的独特向量空间。我们将指纹认证问题建模为评估受害模型空间与嫌疑模型空间之间相似性的任务。针对此问题, 我们引入了两种解决方案: 第一种判断嫌疑输出是否位于受害者的子空间内, 从而实现快速侵权检测; 第二种重建一个联合子空间, 以检测通过参数高效微调 (PEFT) 修改的模型。实验表明, 所提方法在指纹验证方面取得了卓越性能, 并对抗 PEFT 攻击具有鲁棒性。这项工作揭示了 LLMs 的内在特性, 并为保护 LLMs 提供了一种有前景的解决方案, 确保了高效性、通用性和实用性。

## Introduction

大语言模型已成为当代人工智能的基础组成部分, 凭借其生成类人文本和理解复杂语义的能力, 在广泛的自然语言处理任务中展现出卓越性能。尽管其训练需要庞大的数据量和计算资源, 众多开发者仍通过开源发布模型来推动该领域发展。负责诸如 LLaMA、Gemma 和 Mistral 等知名模型的研究团队与组织 [1, 2, 3, 4], 已将其训练有素的大语言模型公开共享, 从而培育了一个充满活力且协作的研究生态系统。这一快速演进的格局在很大程度上由其开源理念所维系。然而, 正是这种加速创新的开放性, 也使这些模型易遭滥用, 例如未经适当署名地利用其他预训练模型进行未授权微调, 或通过虚假所有权声明进行盗用。因此, 保护大语言模型的知识产权不仅对维护其商业价值至关重要, 也对促进该社区的可持续与伦理发展具有迫切意义。

众多研究采用数字水印和指纹技术来保护深度神经网络。例如, Uchida 等人 et al. [5] 引入正则化项以约束网络权重来嵌入水印。随后, 研究者提出了多种方法, 如 [6, 7, 8], 在白盒场景下将水印嵌入模型参数中, 此时提取器可访问全部模型参数。然而, 由于在各种场景中获取全部模型参数存在挑战, 研究重点转向了模型水印的黑盒技术。为此, 许多水印方法利用后门技术, 构建特定的输入-输出映射, 并通过

观察 DNN 模型的输出进行验证, 例如 [9, 10, 11]。生成模型, 特别是图像处理模型, 常产生具有高熵值的内容, 其充足的信息容量可容纳额外的水印信息, 同时保持高度不可感知性。Wu 等人 et al. [12] 引入了一种新框架, 使模型输出包含特定水印。通过从任何含水印的输出中提取水印, 可识别相应 DNN 模型的所有权。Lukas 等人 et al. [13] 提出通过微调图像生成器来嵌入水印, 确保生成的所有图像均带有水印。Fernandez 等人 et al. [14] 将其扩展至扩散模型。通过后门和微调技术嵌入水印会在一定程度上损害模型的主要功能, 且需要大量计算资源。Song 等人 et al. [15] 基于生成模型的伪影和指纹来区分不同模型, 这有助于缓解此问题。大型语言模型 (LLMs) 展现出卓越的推理能力, 但其所需的巨大计算开销为其知识产权保护 (IPP) 带来了新的挑战。Xu et al. [16] 指定一个机密私钥并将其作为指令后门嵌入, 充当指纹。在文献 [17] 中, Zeng et al. 利用 Transformer [18] 的内部参数作为指纹来识别 LLMs。现有方法通常需要白盒访问或微调来验证版权信息。相比之下, 所提出的指纹方法可以在无需任何微调的黑盒场景中实现。

我们提出了一种新颖的 LLM 指纹方法, 通过分析其输出特征来实现 LLM 认证。LLMs 通过从 logits 中采样来生成连贯且上下文恰当的文本, 这些 logits 本身编码了丰富的模型特定信息。在黑盒场景中, LLM 提供商通常通过 API 暴露完整或部分的 logit 向量, 允许用户采用不同的采样策略来生成逼真的内容。Carlini 等人 [19] 已证明, 仅通过此类 API 访问就有可能恢复模型的部分信息。基于这一见解, 我们引入了一个 LLM 指纹框架, 通过从新视角分析模型的 logit 输出来识别每个模型的独特属性。

先前研究已证明, LLMs 的输出位于由其参数决定的线性子空间内 (参见 [20, 21, 22])。在我们的方法中, 我们保留了受害者模型的参数——特别是其最终线性层的参数。通过查询嫌疑模型并收集其输出, 我们利用保留的参数来刻画模型的独特归属。我们将所有权认证形式化为测量受害者模型的向量空间与嫌疑模型输出空间之间相似度的过程。

我们首先介绍一种快速判断给定输出是否源自受害者模型的方法, 通过评估其与保留参数所定义向量空间的兼容性来实现。为了处理涉及参数高效微调 (PEFT) 攻击的场景, 我们进一步设计了一种对齐验证方法, 通过比较它们的表示

相似性来评估嫌疑模型是否是通过 PEFT 从受害者模型衍生而来。此外，在仅能通过 API 访问部分 logits 的情况下，我们证明了可以重建完整的 logits 以促进所有权验证。我们的框架使得仅通过 API 访问即可在黑盒设置中进行模型所有权认证，而无需依赖底层 LLM 的任何特定架构假设。这确保了广泛的适用性，并为 LLMs 的版权保护提供了一个实用且有效的解决方案。实验结果表明，我们的方法在对抗 PEFT 攻击时实现了卓越的验证准确性和鲁棒性，且不损害模型的功能性能。

总而言之，本工作的主要贡献包括：

- 我们从新颖的视角分析了 LLMs 的特性，并证明其输出可作为所有权验证的独特指纹。
- 我们提出了两种互补的 LLM 所有权验证方法：一种能够快速识别输出是否源自受害者模型，另一种则判断嫌疑模型是否是通过 PEFT 攻击从受害者模型衍生而来。
- 我们引入了一种技术，能够从通过 API 访问获取的部分 logits 中重建完整的指纹信息，从而实现在黑盒场景中的所有权验证。

本文的其余结构安排如下。首先，我们概述模型指纹识别、PEFT 以及假定的威胁模型。接着，我们介绍所提出的大语言模型指纹识别技术，随后阐述两种指纹验证方法。之后，提供实验结果与分析。最后，我们对本文进行总结并展开讨论。

## Preliminaries

在本节中，我们简要概述模型指纹识别、参数高效微调（PEFT）以及假定的威胁模型，以便更好地描述所提出的工作。

### Model Fingerprinting

模型指纹识别是一种通过分析深度神经网络（DNN）模型的内在特性来唯一标识和认证模型的技术。每个 DNN 模型，即使是那些架构相同的模型，由于训练数据、初始化和优化过程的差异，都会在其参数和输出中形成独特的模式。通过分析这些模式，例如 logits、嵌入向量或对特定输入的行为响应，模型指纹识别可以生成一个能够区分不同模型的“签名”。该技术支持所有权验证、检测未经授权的副本或微调衍生模型、安全审计等应用，并且通常可以在仅能访问模型输出的黑盒场景中应用。通过这些方法，模型指纹识别为追踪、验证和保护 DNN 模型提供了一种鲁棒且实用的途径。在本研究中，我们利用大语言模型（LLM）的输出作为验证指纹。

### Parameter-Efficient Fine-Tuning (PEFT)

基于预训练模型训练或微调 DNN 需要大量的计算资源，尤其是对于需要大量 GPU 内存的大语言模型（LLM）。因此，近期的研究聚焦于参数高效微调（PEFT）方法，这些

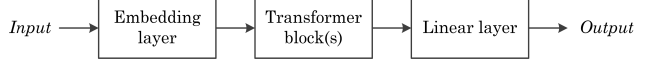


图 1. 基于 Transformer 架构的大语言模型（LLM）流程示意图。

方法能够在仅更新一小部分模型参数的情况下实现有效的优化 [23, 24, 25]。换言之，PEFT 是一套旨在使大型预训练模型适应特定任务而无需更新模型全部参数的技术。

与对整个模型进行微调（这可能计算成本高昂且内存密集）不同，PEFT 通常只修改一小部分参数或添加轻量级模块，同时保持模型的大部分参数冻结。这使得微调过程更快、更便宜且存储效率更高。

低秩适应（LoRA）[24] 已成为 PEFT 的事实标准方法，并作为许多其他方法（如 [25, 26]）的基础。在 LoRA 中，权重矩阵  $W_O$  通过公式  $W_N = W_O + \Delta W = W_O + AB$  进行更新。在训练期间， $W_O$  保持固定，而两个矩阵  $A$  和  $B$  包含最少的可训练参数。在本研究中，我们将使用 LoRA 来模拟 PEFT 攻击。

### Threat Model

我们的威胁模型涉及一个防御者（即模型提供者）和一个控制恶意用户的手。对手的目标是窃取模型并虚假地声称所有权。我们假设对手可能采用 PEFT 攻击（例如 LoRA[24]）来逃避检测。防御者保留对自己模型参数的访问权限，包括作为指纹的最终线性层，其目标是通过 API 访问查询可疑模型来验证所有权。

### LLM Fingerprinting

在本节中，我们将展示大语言模型（LLM）的 logits 输出构成一个向量空间，该空间可用于 LLM 的指纹识别。

### LLM Outputs Span A Vector Space

Transformer 架构因其在各种任务上的卓越性能，已成为众多模型的基础。大语言模型（LLMs）通常是基于 Transformer 的模型，其处理流程如图 1 所示。具体而言，输入文本被分词并转换为词嵌入，嵌入层表示为一个大小为  $|V| \times h$  的矩阵，其中  $|V|$  是词汇表大小， $h$  是隐藏层大小。词汇表大小显著大于隐藏层大小，即  $|V| \gg h$ 。嵌入由包含多个层的 Transformer 块处理，每层包含一个多头自注意力机制和一个前馈层，以计算输入的内在表示并产生中间表示  $z$ 。最后的线性层将  $z$  映射到逻辑值  $s \in \mathbb{R}^{|V|}$ 。LLM 的输出通过对逻辑值  $s$  进行采样生成。为清晰起见，本分析排除了层归一化 [27] 和 RMS 归一化 [28]，因为它们主要引入了额外的乘法项，这些项不影响本文的结论。

值得注意的是，逻辑值  $s = Wz$ ，其中  $W \in \mathbb{R}^{|V| \times h}$  是最后一个线性层的权重矩阵，且  $W$  的秩至多为  $h$ 。由  $W$  产生的

每个输出都对应一个向量  $\mathbf{s}$ ，该向量将始终位于  $\mathbf{R}^{|V|}$  的一个子空间内，该子空间由  $W$  的列张成，其维度至多为  $h$ 。LLM 逻辑值所有可能的输出将张成一个向量空间  $L$ ，该空间与由  $W$  的列张成的空间同构，因为它们具有相同的维度。因此，每个 LLM 都与一个独特的向量空间  $L$  相关联。这种独特性源于向量空间  $\mathbf{R}^{|V|}$  包含了数量极其庞大的可能子空间，对于任何维度为  $h$  的子空间都是如此。例如，在 Gemma [3] 的情况下，给定  $h = 2,048$  和  $|V| = 256,000$ ，子空间的数量是极其庞大的。由于训练初始化、数据集、配置和硬件的差异，两个不同的 LLM 不可能覆盖相同的向量空间。这一特性允许我们将其用作 LLM 的指纹，以识别其所有权。模型提供者只需保留模型中最后一个线性层的参数。通过访问可疑模型的 API，他们可以获取逻辑值，从而验证所有权。

### Vector Space Reconstruction via API

我们已经证明，LLM 输出的逻辑值张成一个由  $L$  表示的向量空间，该空间将被用作 LLM 指纹。获取模型的完整逻辑值并不总是可行的，因为攻击者旨在披露最少的信息以逃避受害者的检测。在本小节中，我们考虑实际场景，在这些场景中，我们的方法重建向量空间以进行指纹验证，并且仅依赖于 API 访问，从而能够获取完整的词汇概率、前  $k$  个概率，或前 1 个概率。

### Complete Probabilities of the Vocabulary

该 API 提供了词汇表上的完整概率分布  $\mathbf{p} = \text{softmax}(\mathbf{s})$ ，其所有分量均为非负且总和为 1。这表明  $\mathbf{p}$  是单纯形  $\Delta^{|V|-1}$  中的一个点，该单纯形位于  $|V|-1$  维子空间内，该子空间属于  $\mathbf{R}^{|V|}$ 。此外， $\mathbf{p}$  还受到  $\mathbf{s}$  的约束。

由于归一化，softmax 函数没有定义良好的逆变换。然而，如果我们暂时忽略它，并使用 CLR（中心对数比）变换，我们可以得到  $\mathbf{s}^*$ ，它与  $\mathbf{s}$  相差一个常数偏差。从空间维度的角度来看，这将引入一个一维的偏差。我们可以手动进行一维校正，并且不会影响结果，因为  $h$  和  $|V|$  都显著大于一。因此，我们可以直接重建与  $\mathbf{s}^*$  对应的向量空间  $L^*$ ，该空间由下式确定：

$$\mathbf{s}^* = \text{CLR}(\mathbf{p}) = \log \left( \frac{\mathbf{p}}{g(\mathbf{p})} \right), \quad (1)$$

其中  $g(\cdot)$  表示输入的几何平均数。我们证明  $\mathbf{s}^*$  与  $\mathbf{s}$  相差一个常数偏差。首先，我们有

$$\mathbf{p} = \text{softmax}(\mathbf{s}) = \frac{e^{\mathbf{s}}}{\sum_{i=1}^{|V|} e^{s_i}} \quad (2)$$

和

$$g(\mathbf{p}) = \left( \prod_{i=1}^{|V|} p_i \right)^{1/|V|}. \quad (3)$$

然后，我们可以发现

$$\begin{aligned} s_i^* &= \log \left( \frac{p_i}{g(\mathbf{p})} \right) = \log(p_i) - \log(g(\mathbf{p})) \\ &= \log(p_i) - \frac{1}{|V|} \sum_{j=1}^{|V|} \log(p_j) \\ &= \log \left( \frac{e^{s_i}}{\sum_{j=1}^{|V|} e^{s_j}} \right) - \frac{1}{|V|} \sum_{j=1}^{|V|} \log \left( \frac{e^{s_j}}{\sum_{k=1}^{|V|} e^{s_k}} \right) \\ &= \left[ s_i - \log \sum_{j=1}^{|V|} e^{s_j} \right] - \frac{1}{|V|} \sum_{j=1}^{|V|} \left[ s_j - \log \sum_{k=1}^{|V|} e^{s_k} \right] \\ &= s_i - \frac{1}{|V|} \sum_{j=1}^{|V|} s_j, \end{aligned} \quad (4)$$

这意味着  $\mathbf{s}^*$  的每个分量仅与  $\mathbf{s}$  的对应分量相差  $\mathbf{s}$  的均值。因此， $\mathbf{s}^* - \mathbf{s}$  构成一个常数向量（偏差）。

### Top- $k$ Probabilities

若 API 提供前  $k$  个概率，即  $\mathbf{p}$  中最大的  $k$  个元素，我们假设提供商允许用户通过 API 使用偏置来修改词元概率。这在应用中是合理且常见的，例如，OpenAI 在其 API 中包含了此功能<sup>1</sup>。该偏置在 softmax 操作前被加到特定词元的 logits 上，API 返回前  $k$  个概率。在不失一般性的情况下，假设  $i_1, i_2, \dots, i_m$  为所选词元的索引且  $b$  为偏置，则偏置后的概率分布  $\mathbf{p}^*$  可通过下式计算：

$$\mathbf{p}^*(i_1, i_2, \dots, i_m, b) = \text{softmax}(\mathbf{s}^*(i_1, i_2, \dots, i_m, b)), \quad (5)$$

其中对于  $1 \leq i \leq |V|$ ，我们有：

$$s_i^*(i_1, i_2, \dots, i_m, b) = \begin{cases} s_i + b & i \in \{i_1, i_2, \dots, i_m\}, \\ s_i & \text{otherwise.} \end{cases} \quad (6)$$

我们提出一种方法来恢复完整的概率  $\mathbf{p} = \text{softmax}(\mathbf{s})$ ，即针对方程 (5) 的  $b \equiv 0$ 。具体而言，通过向模型输入一个提示，我们可以获得前  $k$  个概率，其可表示为  $\{p_{\text{idx}_1}, p_{\text{idx}_2}, \dots, p_{\text{idx}_k}\}$ ，其中  $p_{\text{idx}_1} \geq p_{\text{idx}_2} \geq \dots \geq p_{\text{idx}_k}$  和  $\{\text{idx}_1, \text{idx}_2, \dots, \text{idx}_k\}$  为对应词元的索引。显然，我们有：

$$p_{\text{idx}_i} = \frac{e^{s_{\text{idx}_i}}}{\sum_{j=1}^{|V|} e^{s_j}}, \quad \forall 1 \leq i \leq k. \quad (7)$$

我们已经收集了  $k$  个原始概率  $\{p_{\text{idx}_1}, p_{\text{idx}_2}, \dots, p_{\text{idx}_k}\}$ 。我们需要确定剩余的  $|V| - k$  个原始概率。此问题可通过分批处理这些  $|V| - k$  个词元来解决，每批包含  $k - 1$  个词元。假设我们现在要处理  $k - 1$  个词元，其索引为  $\{i_1, i_2, \dots, i_{k-1}\}$ ，我们向这些  $k - 1$  个词元及第  $\text{idx}_1$  个词元的 logits 添加一个偏置

<sup>1</sup><https://help.openai.com/en/articles/5247780-using-logit-bias-to-alter-token-probability-with-the-openai-api>

$b$ ，以将它们推入“前  $k$  个”。因此，偏置后的概率分布可计算如下：

$$p^*(\text{idx}_1, i_1, \dots, i_{k-1}, b) = \text{softmax}(s^*(\text{idx}_1, i_1, \dots, i_{k-1}, b)), \quad (8)$$

其中对于  $1 \leq i \leq |V|$ ，我们有：

$$s_i^*(\text{idx}_1, i_1, \dots, i_{k-1}, b) = \begin{cases} s_i + b & i \in \{\text{idx}_1, i_1, \dots, i_{k-1}\}, \\ s_i & \text{otherwise.} \end{cases} \quad (9)$$

我们可以写出：

$$p_{i_r} = \text{softmax}(s_{i_r}^*) = \frac{e^{s_{i_r}^*}}{\sum_{j=1}^{|V|} e^{s_j^*}}, \quad \forall i_r \in \{\text{idx}_1, i_1, \dots, i_{k-1}\}, \quad (10)$$

以及

$$\begin{aligned} p_{i_r}^* &= \text{softmax}(s_{i_r}^*) = \frac{e^{s_{i_r}^*}}{\sum_{j=1}^{|V|} e^{s_j^*}} \\ &= \frac{e^{s_{i_r}^*}}{\sum_{j \notin \{\text{idx}_1, i_1, \dots, i_{k-1}\}} e^{s_j^*} + \sum_{j \in \{\text{idx}_1, i_1, \dots, i_{k-1}\}} e^{s_j^*}} \\ &= \frac{e^{s_{i_r}^*}}{e^{s_{i_r}^*} + b} \\ &= \frac{e^{s_{i_r}^*}}{\sum_{j \notin \{\text{idx}_1, i_1, \dots, i_{k-1}\}} e^{s_j^*} + \sum_{j \in \{\text{idx}_1, i_1, \dots, i_{k-1}\}} e^{s_j^* + b}}, \end{aligned} \quad (11)$$

对于任意  $i_r \in \{\text{idx}_1, i_1, \dots, i_{k-1}\}$ 。显然，

$$\frac{p_{\text{idx}_1}}{p_{i_r}} = \frac{e^{s_{\text{idx}_1}}}{e^{s_{i_r}}} \quad (12)$$

且

$$\frac{p_{\text{idx}_1}^*}{p_{i_r}^*} = \frac{e^{s_{\text{idx}_1}^*}}{e^{s_{i_r}^*}} = \frac{e^{s_{\text{idx}_1} + b}}{e^{s_{i_r} + b}} = \frac{e^{s_{\text{idx}_1}}}{e^{s_{i_r}}} \quad (13)$$

因此，对于任意  $i_r \in \{i_1, \dots, i_{k-1}\}$ ，

$$p_{i_r} = p_{\text{idx}_1} \cdot \frac{p_{i_r}^*}{p_{\text{idx}_1}^*}, \quad (14)$$

这表明第  $i_r$  个词元  $p_{i_r}$  的原始概率可以被恢复，因为  $p_{\text{idx}_1}$ 、 $p_{i_r}^*$  和  $p_{\text{idx}_1}^*$  均为已知。通过使用相同提示查询模型，根据上述操作，完整的概率  $p$  可以被完美重构。

需要指出的是，上述操作每次处理  $k-1$  个词元。实际上，我们每次处理少于  $k$  个词元是可行的。因此，我们至少应执行  $(|V|-k)/(k-1)$  次。若  $k-1$  不能整除  $|V|-k$ ，则需要额外再执行一次。此外，每次我们都必须为  $b$  选择一个合适的值，这可以通过启发式方式完成。

### Top-1 Probability

该 API 返回最高概率，这等价于以  $k=1$  获取前  $k$  个概率。针对此场景，我们提出一种方法来恢复完整的概率  $p$ 。

让我们对第  $i$  个标记引入一个显著的偏置  $b$ ，从而将其提升至首位，得到偏置后的概率  $p_i^*$ 。第  $i$  个标记的无偏概率  $p_i$  可通过下式确定：

$$p_i = \frac{1}{e^{b - \log p_i^*} - e^b + 1}. \quad (15)$$

这很容易验证。首先，我们有

$$p_i = \frac{e^{s_i}}{\sum_{j=1}^{|V|} e^{s_j}} \quad (16)$$

和

$$p_i^* = \frac{e^{s_i + b}}{\sum_{j=1, j \neq i}^{|V|} e^{s_j} + e^{s_i + b}}, \quad (17)$$

其可重写为

$$\begin{aligned} p_i^* &= \frac{e^{s_i + b}}{\sum_{j=1}^{|V|} e^{s_j} - e^{s_i} + e^{s_i + b}} \\ &= \frac{e^{s_i} \cdot e^b}{\sum_{j=1}^{|V|} e^{s_j} - e^{s_i} + e^{s_i} \cdot e^b} \\ &= \frac{(p_i \cdot \sum_{j=1}^{|V|} e^{s_j}) \cdot e^b}{\sum_{j=1}^{|V|} e^{s_j} - (p_i \cdot \sum_{j=1}^{|V|} e^{s_j}) + (p_i \cdot \sum_{j=1}^{|V|} e^{s_j}) \cdot e^b} \\ &= \frac{p_i \cdot e^b}{1 - p_i + p_i \cdot e^b} \\ &= \frac{e^b}{p_i^{-1} - 1 + e^b}. \end{aligned} \quad (18)$$

因此，我们得到

$$p_i^{-1} = \frac{e^b}{p_i^*} - e^b + 1. \quad (19)$$

理论上，此方法可在前  $k$  场景中恢复无偏概率，尽管这将需要显著更多的查询次数。然而，由于其涉及指数运算，会引入数值不稳定性并对后续结果产生不利影响，其实用性有限。相比之下，先前描述的方法仅依赖于比例运算，从而在很大程度上避免了数值不稳定性问题。

### Fingerprint Verification

我们介绍了两种用于 LLM 指纹验证的方法。第一种方法涉及验证可疑 LLM 的输出是否与受害 LLM 的输出占据同一空间。这意味着可疑模型与受害模型共享相同的最后一个线性层，便于快速识别模型侵权。如果模型经过微调，表明最后一个线性层的参数已被修改，即向量空间发生变化，这使得验证变得困难。为了解决这个问题，我们提出了一种对齐验证方法来应对这一挑战。该方法计算由可疑模型的输出向量空间与受害模型的参数空间共同形成的联合空间维度。通过比较这两个空间的相似性来识别模型侵权。图 2 展示了 LLM 指纹验证的示意图。

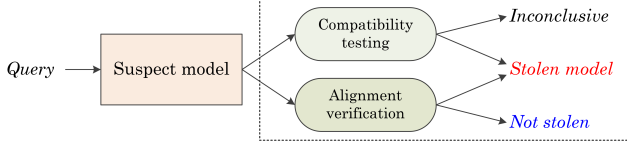


图 2. LLM 指纹验证示意图。

### Compatibility Testing

如上所示，LLM 的输出张成一个向量空间  $L$ ，该空间与最后一个线性层  $W$  的列所张成的空间同构。在接下来的讨论中，我们略微滥用记号，使用  $L$  来表示由最后一个线性层  $W$  的列所张成的空间。我们将 LLM 的最后一个线性层保留为私有，作为受害者模型的指纹。通过 API 查询嫌疑模型以获得其 logits 输出  $s$ 。

对于源自受害者模型且未修改最后一个线性层的嫌疑模型，其 logits 预计位于  $L$  内。这可以通过尝试求解线性系统  $Wx = s$  来形式化验证，以确定  $s$  是否属于  $L$ 。若解存在，则认为嫌疑模型源自受害者模型。实际上，由于浮点计算引入的数值误差，我们改为计算  $s$  与子空间  $L$  之间的欧几里得距离  $d$ ，计算公式为：

$$d = \|s - W\hat{x}\|, \quad (20)$$

其中  $\hat{x}$  表示  $Wx = s$  的解。令  $e$  表示由数值不稳定性引起的误差容限。如果  $d < e$ ，我们则得出结论： $s$  与  $L$  相容，表明嫌疑模型很可能源自受害者模型。

当仅能访问输出概率  $p$ ，而 logits $^*$  是使用先前描述的方法重建时，我们通过向  $W$  附加一个常数列向量来考虑固有的一维偏差，从而得到  $W^* = [W, \mathbf{1}]$ 。随后使用  $W^*$  替代  $W$  进行验证，遵循上述相同的方法论。

### Alignment Verification

由于 PEFT 会改变模型参数的一部分，上述描述的验证方法便无法再可靠地应用。根据流形假设，高维数据可以映射到一个低维潜在流形上，在该流形内，不同的任务由不同的特征方向控制。因此，当针对特定任务进行微调时，假定其仅影响向量空间  $L$  的一部分，而非整个空间。特别是在 PEFT 攻击中，模型使用一个低秩矩阵进行微调。原始矩阵  $W_O$  通过

$$W_N = W_O + \Delta W, \quad (21)$$

进行更新，其中  $\Delta W$  是一个秩不超过  $k$  的矩阵。如果可疑模型的输出空间与受害模型  $W$  的列所张成的空间之间存在显著重叠，则可疑模型与受害模型极为相似，可能源自未经授权的复制。

我们计算可疑模型的向量空间与受害模型参数空间的并集所形成的维度，记为  $L_{\text{sum}}$ 。如果  $L_{\text{sum}}$  的维度与受害模型

### Algorithm 1 Pseudocode for dimension difference calculation

Input: The parameter matrix  $W$ , the logits set  $S = \{s_1, s_2, \dots, s_q\}$ , the error term  $e_r$ .

Output: The dimension difference  $\Delta r$ .

```

1: Initiate  $\Delta r = 0$  and  $W_{\text{sum}} = W$ 
2: for  $i = 1, 2, \dots, q$  do
3:   Solve  $W_{\text{sum}} \cdot x_i = s_i$  to obtain  $\hat{x}_i$ 
4:   Calculate  $d_i = \|s_i - W_{\text{sum}} \cdot \hat{x}_i\|$ 
5:   if  $d_i > e_r$  then
6:      $\Delta r = \Delta r + 1$ 
7:      $W_{\text{sum}} = [W_{\text{sum}}, s_i]$ 
8:   end if
9: end for
10: return  $\Delta r$ 

```

空间的维度非常接近，则表明可疑模型源自受害模型。否则，便不是。这可以通过设定一个阈值来控制。对于那些包含大量浮点数的矩阵，直接计算其秩并不可取，因为数值误差会导致显著错误。相反，我们在此提出计算  $L_{\text{sum}}$  与  $L$ （对应于  $W$ ）之间的维度差异  $\Delta r$ ，以判断可疑模型是否源自受害模型。如果  $\Delta r$  小于一个较小的阈值（或者说  $\Delta r$  显著小于隐藏大小  $h$ ），则意味着强对齐，并支持可疑模型源自受害模型的推断。在仅能访问输出概率的场景中， $\Delta r$  只会经历数值上的单位扰动，这不会影响我们的结果。算法 1 给出了维度差异计算的伪代码。

### Experimental Results and Analysis

在本节中，我们将报告实验结果并提供分析，以证明我们工作的适用性。

#### Setup

我们使用隐藏层大小为 2048 的 Gemma [3] 作为受害模型，并通过 LoRA [24] 或 QLoRA [25] 进行微调作为对抗设置。我们将秩设置为 16、32 和 64。模型在 Alpaca 数据集 [29] 和 SAMSum 数据集 [30] 上进行微调，以模拟不同场景。我们还与一个称为 Gemma-2 的新版本进行比较，该版本结构相同但采用新颖的训练方法，导致对相同输入产生显著的收入和完全不同的输出。对于每个实验，我们生成（或恢复）300 个完整的逻辑值（或概率）用于模拟。误差参数  $e_r$  在所有实验中固定为 1。以下小节报告兼容性测试和对齐验证的结果，随后是一个案例研究，说明我们方法的实际适用性<sup>2</sup>。

<sup>2</sup>代码地址：<https://github.com/solitude-alive/llm-fingerprint>

表 1. 在不同模型版本以及使用不同模块和秩进行微调的模型上的兼容性测试结果。

| Model     | Dataset | Scenario             |                      |                      |                      |
|-----------|---------|----------------------|----------------------|----------------------|----------------------|
|           |         | Full logits          | Full probabilities   | Top-5 probabilities  | Top-1 probabilities  |
| Gemma     | /       | $8.0 \times 10^{-5}$ | $3.4 \times 10^{-3}$ | $3.4 \times 10^{-3}$ | $8.3 \times 10^{-5}$ |
| Gemma-2   |         | $5.5 \times 10^5$    | $5.5 \times 10^5$    | $5.5 \times 10^5$    | $5.5 \times 10^5$    |
| qkv-16    | SAMSum  | $9.7 \times 10^{-5}$ | $1.4 \times 10^{-4}$ | $1.4 \times 10^{-4}$ | $1.0 \times 10^{-4}$ |
| qkv-32    |         | $8.5 \times 10^{-5}$ | $3.6 \times 10^{-3}$ | $3.6 \times 10^{-3}$ | $9.6 \times 10^{-5}$ |
| qkv-64    |         | $1.1 \times 10^{-4}$ | $9.5 \times 10^{-4}$ | $9.5 \times 10^{-4}$ | $9.9 \times 10^{-5}$ |
| linear-16 |         | $7.3 \times 10^5$    | $7.2 \times 10^5$    | $7.2 \times 10^5$    | $7.2 \times 10^5$    |
| linear-32 |         | $6.9 \times 10^5$    | $6.8 \times 10^5$    | $6.8 \times 10^5$    | $6.9 \times 10^5$    |
| linear-64 |         | $7.6 \times 10^5$    | $7.5 \times 10^5$    | $7.5 \times 10^5$    | $7.5 \times 10^5$    |
| qkv-16    | Alpaca  | $8.3 \times 10^{-5}$ | $5.7 \times 10^{-4}$ | $5.7 \times 10^{-4}$ | $8.4 \times 10^{-4}$ |
| qkv-32    |         | $7.0 \times 10^{-5}$ | $2.8 \times 10^{-3}$ | $2.8 \times 10^{-3}$ | $8.5 \times 10^{-5}$ |
| qkv-64    |         | $6.8 \times 10^{-5}$ | $1.6 \times 10^{-4}$ | $1.6 \times 10^{-4}$ | $8.1 \times 10^{-5}$ |
| linear-16 |         | $6.8 \times 10^5$    | $6.7 \times 10^5$    | $6.7 \times 10^5$    | $6.7 \times 10^5$    |
| linear-32 |         | $6.7 \times 10^5$    | $6.6 \times 10^5$    | $6.6 \times 10^5$    | $6.6 \times 10^5$    |
| linear-64 |         | $6.9 \times 10^5$    | $6.8 \times 10^5$    | $6.8 \times 10^5$    | $6.9 \times 10^5$    |

表 2. 在不同模型版本以及使用不同模块和秩进行微调的模型上的维度差异结果。

| Model     | Dataset | Scenario    |                    |                     |                     |
|-----------|---------|-------------|--------------------|---------------------|---------------------|
|           |         | Full logits | Full probabilities | Top-5 probabilities | Top-1 probabilities |
| Gemma     | /       | 0           | 1                  | 1                   | 1                   |
| Gemma-2   |         | 300         | 300                | 300                 | 300                 |
| qkv-16    | SAMSum  | 0           | 1                  | 1                   | 1                   |
| qkv-32    |         | 0           | 1                  | 1                   | 1                   |
| qkv-64    |         | 0           | 1                  | 1                   | 1                   |
| linear-16 |         | 10          | 11                 | 11                  | 11                  |
| linear-32 |         | 11          | 12                 | 12                  | 11                  |
| linear-64 |         | 10          | 11                 | 11                  | 11                  |
| qkv-16    | Alpaca  | 0           | 1                  | 1                   | 1                   |
| qkv-32    |         | 0           | 1                  | 1                   | 1                   |
| qkv-64    |         | 0           | 1                  | 1                   | 1                   |
| linear-16 |         | 15          | 16                 | 16                  | 16                  |
| linear-32 |         | 20          | 21                 | 21                  | 20                  |
| linear-64 |         | 21          | 22                 | 22                  | 21                  |

Compatibility Testing Results

在兼容性测试中，我们假设恶意用户可能以原始形式或经过微调后发布其窃取的模型。微调应用于中间层的注意力机制或最后的线性层。在此，我们保留模型最后一层线性层的参数，并在多种场景下进行验证实验。平均欧几里得距离  $d$  被用来量化性能。如表 1 所示，每一列对应不同 API 场景下

$d$  的结果，而每一行代表不同模型的结果。‘qkv’ 和 ‘linear’ 表示 LoRA 模块被应用于注意力机制中的查询、键和值模块或最后一层的线性模块，其后缀表示 LoRA 的秩。实验结果清晰地表明，对于在最后一层微调或未微调的模型，它们与其他模型之间存在显著差异。这种差异足以用于确定所有权，验证了所提方法的有效性。值得注意的是，尽管这是平均结

表 3. 在不同数据集上使用不同秩的 QLoRA 进行微调的模型维度差异结果。

| Model     | Dataset | Scenario    |                    |                     |                     |
|-----------|---------|-------------|--------------------|---------------------|---------------------|
|           |         | Full logits | Full probabilities | Top-5 probabilities | Top-1 probabilities |
| Gemma     | /       | 0           | 1                  | 1                   | 1                   |
| Gemma-2   |         | 300         | 300                | 300                 | 300                 |
| qkv-16    | SAMSum  | 0           | 1                  | 1                   | 1                   |
| qkv-32    |         | 0           | 1                  | 1                   | 1                   |
| qkv-64    |         | 0           | 1                  | 1                   | 1                   |
| linear-16 |         | 9           | 10                 | 10                  | 10                  |
| linear-32 |         | 10          | 11                 | 11                  | 11                  |
| linear-64 |         | 8           | 9                  | 9                   | 9                   |
| qkv-16    | Alpaca  | 0           | 1                  | 1                   | 1                   |
| qkv-32    |         | 0           | 1                  | 1                   | 1                   |
| qkv-64    |         | 0           | 1                  | 1                   | 1                   |
| linear-16 |         | 16          | 17                 | 17                  | 17                  |
| linear-32 |         | 20          | 21                 | 21                  | 21                  |
| linear-64 |         | 23          | 24                 | 24                  | 24                  |

果，但在实践中即使只有少量样本，这种一致性依然成立，因此也能实现快速验证。

### Alignment Verification Results

在对齐验证方面，表 2 展示了维度差异计算的结果，即  $\Delta r$ 。PEFT 攻击的配置与前一节相同。如表 2 所示，所提方法在所有四种 API 场景下均能一致地区分不同类型的微调。对于在注意力模块上微调的模型，该度量返回的  $\Delta r$  始终很小，表明与原始模型具有很强的一致性。相比之下，在最终线性层上微调的模型则表现出显著更大的  $\Delta r$ ，且该值随 LoRA 秩的增加而系统性增长。所有这些观察结果不仅突显了 qkv 微调与线性微调之间的差异，更直接证明了我们方法的判别能力。无论微调位置、微调强度或 API 输出的粒度如何，我们的方法都能可靠地捕捉 PEFT 引入的结构偏差，从而有效识别被窃取或微调的变体。在所有 API 场景中观察到的一致趋势进一步验证了所提验证方法的鲁棒性和样本效率。表 3 报告了在不同数据集上使用不同秩的 QLoRA 微调的模型的维度差异结果。从该表中，我们可以得出相同的结论。

值得注意的是，表 2 和表 3 中为 Gemma-2 报告的  $\Delta r = 300$  是由于我们仅使用了 300 个有效的完整 logits（或概率）进行模拟的结果。如果放宽此限制并包含更多输出，观察到的  $\Delta r$  可能会成比例增加。事实上，对 Gemma-2 的 3000 个输出进行分析得到  $\Delta r = 2052$ ，这表明存在广泛的微调甚至是从头训练。这进一步强调了所提指纹的唯一性和可靠性。

### Case Study

我们对五个共享相同架构的 Llama 系列模型进行了案例研究，以进一步评估我们的指纹方法在高模型相似性现实条件下的有效性。在图 3 中，通过欧几里得距离度量计算并经过对数变换后可视化的贡献对数与权重呈现出清晰且稳定的模式，即只有对角线项显示出显著升高的值，而所有非对角线项均保持统一低位。这表明我们的度量方法始终将每个模型识别为与自身最兼容，并有效拒绝了所有其他密切相关的变体。关键在于，这种显著的对角线优势并非源于细微的架构差异，而是直接反映了我们方法的判别强度与可靠性。即使在架构完全相同且训练目标高度相似的模型之间，生成的指纹依然具有区分度且保持稳定，从而在实际应用中实现了鲁棒且可信的模型归属判定。

### Conclusion and Discussion

本文提出了一种新颖且通用的大语言模型指纹框架，能够在无需重新训练或修改目标模型的黑盒设置下实现可靠的所有权验证。在多个模型系列、API 粒度以及 PEFT 攻击配置的实验中，我们的方法均能生成具有区分度且稳定的指纹。该方法持续实现高验证准确率，并对不同模块的 LoRA 微调保持鲁棒性。这些发现突显了大语言模型中存在的持久性结构特性，即使在对抗条件下也能实现可靠的模型归属判定。总体而言，本研究为大语言模型的溯源追踪与所有权保护提供了一种实用且有效的解决方案，并为生成模型的安全部署指明了未来研究的可行方向。

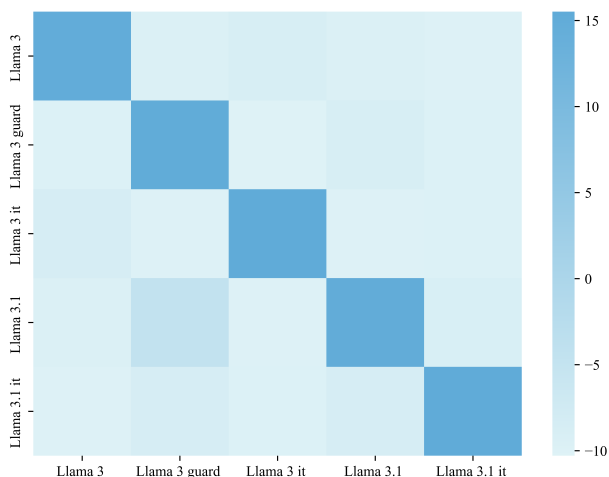


图 3. Llama 系列模型的案例研究。

## Acknowledgment

本研究获得以下项目资助：南宁市“邕江”计划（项目编号 RC20250102）、上海市科学技术委员会（项目编号 24ZR1424000）以及西藏自治区中央引导地方科技发展基金项目（项目编号 XZ202401YD0015）。

## 参考文献

- [1] H. Touvron, T. Lavril, G. Izacard et al. LLaMA: 开放且高效的基础语言模型. arXiv preprint arXiv:2302.13971, 2023.
- [2] H. Touvron, L. Martin, K. Stone et al. Llama 2: 开放基础模型与微调对话模型. arXiv preprint arXiv:2307.09288, 2023.
- [3] T. Mesnard, C. Hardin, R. Dadashi et al. Gemma: 基于 Gemini 研究与技术的开放模型. arXiv preprint arXiv:2403.08295, 2024.
- [4] A. Jiang, A. Sablayrolles, A. Mensch et al. Mistral 7B. arXiv preprint arXiv:2310.06825, 2023.
- [5] Y. Uchida, Y. Nagai, S. Sakazawa, S. Satoh. 将水印嵌入深度神经网络. 见于: Proc. ACM International Conference on Multimedia Retrieval, 第 269-277 页, 2017.
- [6] J. Wang, H. Wu, X. Zhang, Y. Yao. 通过误差反向传播在深度神经网络中嵌入水印. 见于: Proc. IS&T Electronic Imaging, Media Watermarking, Security and Forensics, 第 22-1-22-9(9) 页, 2020.
- [7] B. Rouhani, H. Chen, F. Koushanfar. DeepSigns: 一个用于深度神经网络所有权保护的端到端水印框架. 见于: Proc. International Conference on Architectural Support for Programming Languages and Operating Systems, 第 485-497 页, 2019.
- [8] P. Fernandez, G. Couairon, T. Furon, M. Douze. 用于水印大型 Transformer 的函数不变量. 见于: Proc. IEEE International

- Conference on Acoustics, Speech and Signal Processing, 第 4815-4819 页, 2024.
- [9] Y. Adi, C. Baum, M. Cisse et al. 化弱点为优势: 通过后门植入为深度神经网络添加水印. 见于: Proc. USENIX Conference on Security Symposium, 第 1615-1631 页, 2018.
- [10] X. Zhao, H. Wu, X. Zhang. 通过随机图对神经网络进行水印处理. 见于: Proc. IEEE International Symposium on Digital Forensics and Security, 第 1-6 页, 2021.
- [11] Y. Liu, H. Wu, X. Zhang. 基于傅里叶扰动分析和频率敏感性聚类的鲁棒且不可感知的黑盒 DNN 水印技术. IEEE Transactions on Dependable and Secure Computing, 第 21 卷, 第 6 期, 第 5766-5780 页, 2024.
- [12] H. Wu, G. Liu, Y. Yao, X. Zhang. 使用带水印图像为神经网络添加水印. IEEE Transactions on Circuits and Systems for Video Technology, 第 31 卷, 第 7 期, 第 2591-2601 页, 2021.
- [13] N. Lukas, F. Kerschbaum. PTW: 用于预训练图像生成器的关键调谐水印技术. 见于: Proc. USENIX Conference on Security Symposium, 第 2241-2258 页, 2023.
- [14] P. Fernandez, G. Couairon, H. Jégou et al. 稳定签名: 将水印根植于潜在扩散模型中. 见于: IEEE International Conference on Computer Vision, 第 22466-22477 页, 2023.
- [15] H. Song, M. Khayatkhoei, W. AbdAlmageed. ManiFPT: 定义与分析生成模型的指纹. 见于: Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition, 第 10971-10981 页, 2024.
- [16] J. Xu, F. Wang, M. Ma et al. 大语言模型的指令指纹识别. 载于: Proc. NAACL, 第 3277-3306 页, 2024 年。
- [17] B. Zeng, L. Wang, Y. Hu et al. HuRef: 大语言模型的人类可读指纹. 载于: Proc. Neural Information Processing Systems (NeurIPS), 2024 年。
- [18] A. Vaswani, N. Shazeer, N. Parmar et al. 注意力机制就是你所需要的一切. 载于: Proc. Neural Information Processing Systems (NeurIPS), 2017 年。
- [19] N. Carlini, D. Paleka, K. D. Dvijotham et al. 窃取生产语言模型的一部分. 载于: Proc. International Conference on Machine Learning, 第 5680-5705 页, 2024 年。
- [20] Z. Yang, Z. Dai, R. Salakhutdinov et al. 打破 Softmax 瓶颈: 一种高秩 RNN 语言模型. 载于: Proc. International Conference on Learning Representations, 2018 年。
- [21] M. Finlayson, J. Hewitt, A. Koller et al. 终结神经文本退化的奇案. 载于: Proc. International Conference on Learning Representations, 2024 年。
- [22] M. Finlayson, X. Ren, S. Swayamdipta. 受 API 保护的大语言模型的逻辑值泄露专有信息. 载于: Proc. Conference on Language Model, 2024 年。



- [23] N. Houlsby, A. Giurui, S. Jastrzebski et al. 自然语言处理的参数高效迁移学习。载于: Proc. International Conference on Machine Learning, 第 2790-2799 页, 2019 年。
- [24] E. Hu, Y. Shen, P. Wallis et al. LoRA: 大语言模型的低秩自适应。载于: Proc. International Conference on Learning Representations, 2022 年。
- [25] T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer. QLoRA: 量化大语言模型的高效微调。载于: Proc. Neural Information Processing Systems (NeurIPS), 2023 年。
- [26] F. Meng, Z. Wang, M. Zhang. PiSSA: 大语言模型的主奇异值与奇异向量自适应。载于: Proc. Neural Information Processing Systems (NeurIPS), 2024 年。
- [27] J. Ba, J. Kiros, G. E. Hinton. 层归一化。arXiv preprint arXiv:1607.06450, 2016 年。
- [28] B. Zhang, R. Sennrich. 均方根层归一化。载于: Proc. Neural Information Processing Systems (NeurIPS), 2019 年。
- [29] Y. Dubois, X. Li, R. Taori et al. AlpacaFarm: 一种用于从人类反馈中学习方法模拟框架。载于: Proc. Neural Information

Processing Systems (NeurIPS), 2023 年。

- [30] B. Gliwa, I. Mochol, M. Biesek, A. Wawer. SAMSum 语料库: 一个用于抽象摘要的人类标注对话数据集。载于: Proc. 2nd Workshop on New Frontiers in Summarization, 2019 年。

## Author Biography

杨志广于 2022 年和 2025 年分别在中国上海大学获得学士和硕士学位。他的研究兴趣包括深度学习、大语言模型、数字水印与指纹识别。他目前在一家初创公司担任算法工程师。吴汉舟分别于 2011 年和 2017 年在中国成都的西南交通大学获得学士和博士学位。2014 年至 2016 年期间, 他在美国新泽西州的新泽西理工学院担任访问学者。2017 年至 2019 年, 他在中国北京的中国科学院自动化研究所担任研究科学家。他现为上海上海大学的副教授。他的研究兴趣包括隐写术、隐写分析、数字水印和数字取证。他已在同行评审的期刊和会议上发表了 100 多篇研究文章。他还撰写了四个书籍章节。他曾担任 2022 年 IEEE 信息取证与安全国际研讨会的组织主席, 并自 2025 年起担任 IEEE 信号处理快报的副编辑。